Lctr 14: Mixed C and ASM Programming

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- ARM Cortex-M core registers.
- ARM Cortex-M ISA (instruction set architecture).
- EABI (embedded application binary interface) standard for creating ASM functions.
- Using the stack in ASM functions.

Demo project:

- Mixed C and ASM programming.
- Calling ASM functions in C.
- Returning from a function
  - `bx lr` when LR is not stacked; or
  - `pop pc` when LR is stacked.

14.1 Introduction

14.1.1 Reasons to learning ASM programming

14.1.2 Mixed C and ASM programming

There are four cases of mixed C and ASM programming:

- A C program calls ASM functions defined in a `.s` file.
- A C program calls ASM functions defined in a `.c` file.
- Embed inline ASM instructions in C functions.
- An ASM program defined in a `.s` file calls C functions defined in a `.c` file.

We will focus on writing ASM functions in separate .s files that can be called in C.

14.1.3 GNU ASM vs ARMASM

Here are the rules and conventions we follow when using ASM:

- When developing a project with CubeIDE, we use the GNU syntax. Here, we use **lowercase** letters for all the instructions and registers. This applies to the embedded code in our lectures.
- When reading the documentations produced by ARM, we need to use the ARMASM syntax. Here, we need to use **capital case** letters for all the instructions and registers. This applies to the text of our lectures.

14.2 ARM Cortex-M architecture and core registers

14.2.1 ARM Cortex-M architecture

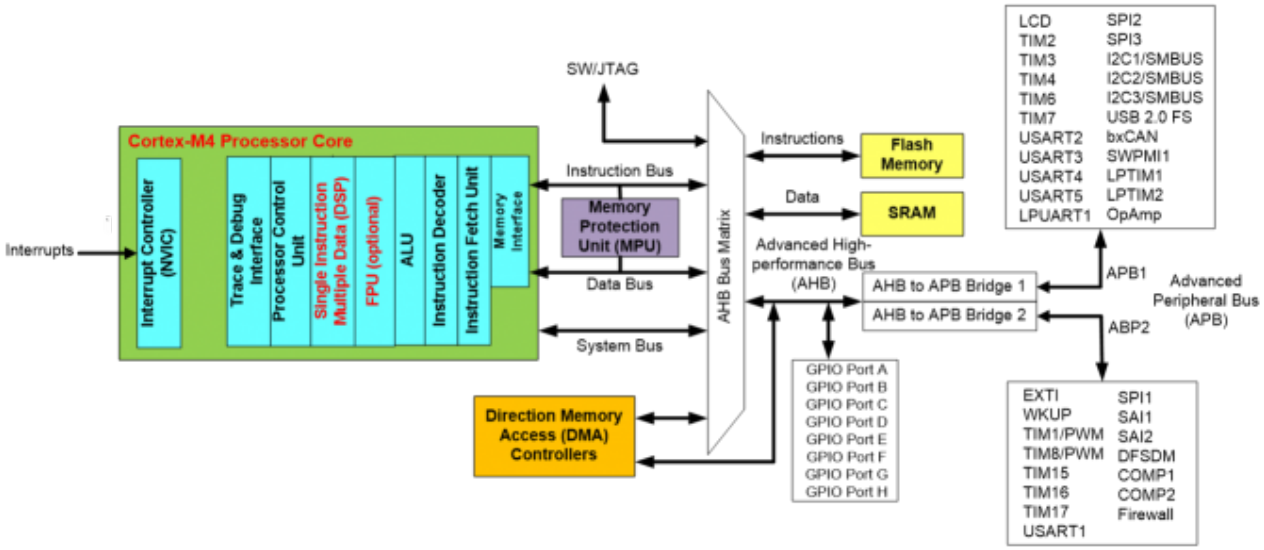

Figure 1: A simplified hardware organization of a Cortex-M4 MCU.

14.2.2 ARM Cortex-M core registers

A Cortex-M core has the following *common* registers:

- R0 to R12: general-purpose registers. These registers are used to hold the operation operands or results for various operations. They are also used to hold addresses of memories, like the pointers in C, used for memory access.
  - R0 to R7 are called low registers. (Some 16-bit instructions can only use low registers due to the limited size of op-code, the encoding in the ASM instructions.)
  - R8 to R12 are called high registers. (Some of these registers have some specially designated usages; we will not discuss the details here.)
- SP (R13): stack pointer. There are actually two stack pointers. Which of these two is used is determined by a bit in the Control register, as will be discussed later.
  - PSP: the process stack pointer. This is used by the user processes (also called tasks or threads) when an RTOS (real-time operating system) is used.

  - MSP: the main stack pointer. This is used by the interrupt handlers when an RTOS is used. If no RTOS is used, the super loop of the main program uses this stack pointer.

- LR (R14): link register. LR is used to hold the address a function needs to return to when it runs to the end.
- PC (R15): program counter. PC is used to hold the address of the instruction to be executed next.

14.3 ARM Cortex-M ISA (instruction set architecture)

14.3.1 Thumb ISA with Thumb-2 technology

ARM has designed different processors which use different ISAs, and Cortex-M processors use *Thumb* instructions with Thumb-2 technology.

14.3.2 Generic format of Cortex-M instructions

The generic syntax of a Cortex-M instruction is shown below

```
a_label:
    add     r0, r1, r2  @ r0 = r1 + r2
```

The various parts are described below:

- `a_label` is a label to which any *branch* instructions can jump to. If there is no need to jump to this line of . There is no need to use this label if this instruction is not intended be jumped to. Yet, if we do use this label, *there should be no white space before the label*.
- `add` is the mnemonic, the instruction. This specific one is for addition and will be written as ADD in the text. Note that we need to have white spaces before the mnemonic. The mnemonic can be on the same line as the label if there is one, but more often it is on the next line to make the code easier to read.
- `r0` is R0, the register holding the **destination operand** of the addition. **This can be omitted** if it is the same register as the **first source operand**.
- `r1` is R1, the register holding the first source operand of the addition.
- `r2` is R2, the **second source operand** of the addition. While the above two operands have to be registers, *this second operand can be an immediate number or a shifted version of a register*.

This is called **flexible operand 2**.

- `@ r0= r1+ r2` is the comment. We can also use the C-style comments as well.

14.3.3 Classification of Cortex-M instructions

The Cortex-M instruction set can be classified into the following:

- Arithmetic and logic: Add, subtract, multiply, divide, shift, rotate, and various bitwise (logic) operations.
  - We use only general-purpose registers and immediate numbers for these instructions.
- Data movement: Load, store, move.
  - Load and store instructions exchange data between registers and the memories.
  - Move instructions exchange data between registers or load a (small) immediate number to a register.
  - We often use the sequence of **load, modify, and store** operations in various programming.
- Compare and branch: Compare, test, if-then, and compare-and-branch-on-zero.
- Floating-point arithmetic.
- Miscellaneous: Breakpoints, wait for events, interrupt enable/disable, data memory barrier, data synchronization barrier.

14.4 The EABI standard for argument passing and value return

In mixed C and ASM programing, we often call a function written in ASM from C. When we call a (C or an ASM) function, the arguments are passed via registers, as prescribed in the EABI (Embedded Application Binary Interface) standard, which also prescribes how the return value is passed back to the caller.

14.4.1 Passing arguments to and returning value from a function

- **R0 to R3** are used to pass up to four 32-bit arguments to a function. Note that if the arguments have data types less than 32 bits, they are type-casted to 32-bit ones to be passed to these registers. **Argument 1 will be passed to R0, and Argument 2 to R1**, and so on.

- **R0 is used to return a 32-bit value from the function**. Similarly, if the return value is less than 32 bits, we just need to put the value in the lower halfword or the least significant byte of R0.
- If we use 64-bit arguments (specified by parameter definition), we need to use two registers to hold the value of a single argument. We still use the *little-endian* principle—using R0 to hold the lower word and R1 the upper one.
- If we return a 64-bit value, we need to use R0 and R1 to hold the return value following the above little-endian principle.

Here are a few points to consider:

- We only consider the case of using integer arguments in function calls in our projects if not specifically indicated otherwise.
- We don't consider the cases that have more than four arguments for ASM function programming. If we do have more data to pass to a function, we can use a `struct` to pack the data together and pass its pointer.
- Since the caller, according to EABI, can use R0 to R3 to pass the arguments to the functions, the contents of these registers do not need to be preserved within the function for the caller; it is the caller's responsibility to preserve these registers if needed, as will be shown below. Yet, **if the function uses other registers, such as R4 and R5, it is the function's responsibility to preserve these registers for the caller, as illustrated below**.

14.4.2 Returning from an ASM function

Call a function and return:

- PC is used to point to the next address of execution.
- When calling a function, PC points to the function. The address of the statement following function call is stored in LR.
- When returning from the function, we need to move the value of LR to PC, which can be done using the `bx lr` instruction.

```
    int casted = mp_add_in_s(s_8, 0);
8003308:   f997 301f   ldrsb.w r3, [r7, #31]
800330c:   2100        movs    r1, #0
800330e:   4618        mov r0, r3
8003310:   f7fc ff7e   bl  8000210 <mp_add_in_s>
8003314:   6138        str r0, [r7, #16]
```

There is another approach to return from the function, which is related to preserving the environment for the caller, as discussed below.

14.5 Using the stack in a function

14.5.1 Using the stack to preserve environment for the caller

The Cortex-M processors use a *full descending* **stack**, as illustrated in Figure 2 (Fig 4.12 on page 101 of [ref_Yiu_m0]):

- *Full* means that the stack pointer always points to the **last** pushed-in data, not an empty one.
- *Descending* means that when we push new data, we need to **reduce** the value of the SP (by 4 as we preserve 4 bytes); then we preserve the contents of the registers in the new locations in the stack.

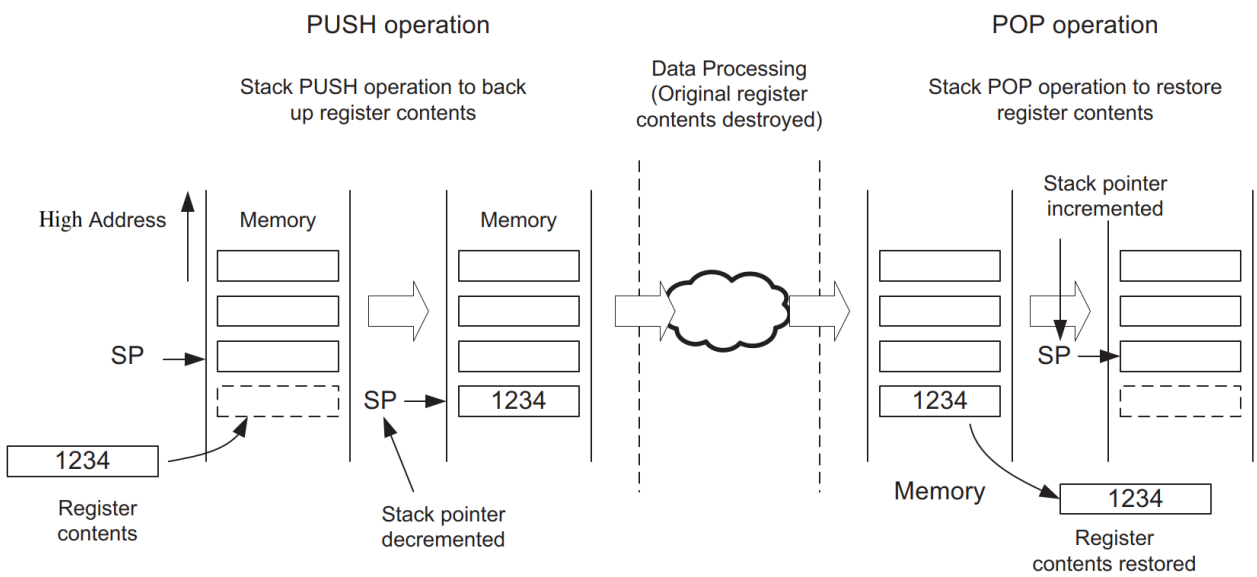

Figure 2: Illustration of a full descending stack.

Assume we are in a function and the value of SP is 0x2000_1008. Also assume the value of R4 is 1234, which is used by the caller. Now, assume we have to use R4 for an operation in the ASM function. Here, as described in the comments in the code below, we need to use the PUSH (save from register to stack) and POP (load from stack to register) instructions to preserve the value of R4 for the caller.

```
    push  {r4}   @ Done before R4 is used
    @ Now, SP points to 0x2000_1004, as it decrements by 4.
    @ The value pointed by SP is 1234.
    @
    @ Perform other processing needed in the function.
    @
    pop   {r4}   @ Done before returning to the caller.
    @ Now, SP points to 0x2000_1008, as it increments by 4.
    @ The value of R4 is restored to be 1234.
```

Note that we can push multiple registers with one instruction, using the syntax below:

```
    push {r6, r4, r8-r10, lr}
```

With the above instruction, we push the following registers in the stack: R4, R6, R8, R9, R10, and LR. Note that when we use the PUSH instruction to preserve multiple registers, we will **always store the lower-numbered registers in the memory locations with lower addresses**, regardless the order

they are in the curly braces. This is the reason that we need to number SP (R13), LR (R14), and PC (R15). For example, if SP points to 0x2000_1020 before the above PUSH instruction, we will have the followings addresses in the stack for the above registers after running this instruction:

Address	Register
0x2000_101C	LR
0x2000_1018	R10
0x2000_1014	R9
0x2000_1010	R8
0x2000_100C	R6
0x2000_1008	R4

Note that

```
pop {r4, r6, r8-r10, lr}
bx  lr
```

is equivalent to

```
pop {r6, r4, r8-r10, pc}
```

as we have effectively put the value of LR back to PC. The later is preferred, and **this is the second approach to return to the caller**.

14.5.2 Using the stack for local variables

14.6 Demo project

The demo project, `fc1s_mixed_c_n_asm_add`, is a mixed C and ASM programming project. In this project, we illustrate how to write addition functions in ASM, saved in a `.s` file named `fc1s_asm_add_in_s.s`.

These ASM functions are called by C functions in `fc1s_mixed_c_n_asm_add_app.c`.

14.6.1 ASM functions in a .s file

We have three ASM function in `fc1s_asm_add_in_s.s`. The instructions and directives are commented in detail in the code. So, these functions should be readable even though we have not discussed the instructions yet.

```
    .section .text      @ Define the program section
    .syntax unified     @ Specify to use Thumb-2 instructions

    .align              @ Align to memory boundary, always use it

    .global mp_add_in_s             @ Define a global variable
    .type   mp_add_in_s, %function  @ Indicate this is a function

    .global mp_add_in_s_inefficient
    .type   mp_add_in_s_inefficient, %function

    .global mp_add_in_s_bad_return
    .type   mp_add_in_s_bad_return, %function

@ int mp_add_in_s(int x, int y)
@   Input:
@     * x -> r0
@     * y -> r1
@   Output:
@     * r0
mp_add_in_s:            @ Start address of func mp_add_in_s
    .fnstart            @ Optional
    add     r0, r0, r1  @ R0 = R0 + R1, R0 holds return value
    bx      lr          @ LR -> PC, return from the function
    .fnend              @ Optional

@ int mp_add_in_s_inefficient(int x, int y)
@   Input and output: same as int mp_add_in_s(int x, int y)
mp_add_in_s_inefficient:    @ An inefficient addition function
    push    {r4, lr}    @ Will use R4 later; push it to stack
                        @ LR is pushed to stack as well
    add     r4, r0, r1  @ R4 = R0 + R1
    mov     r0, r4      @ Move content of R4 to R0
    pop     {r4, pc}    @ Pop the stacked value of R4 to R4
                        @ LR -> PC, return from the function
```

```

@ int mp_add_in_s_bad_return(int x, int y);
@   Input and output: same as int mp_add_in_s(int x, int y)
mp_add_in_s_bad_return:
    push    {r4, lr}
    add     r4, r0, r1
    mov     r0, r4
    pop     {r4, lr}    @ Fail to have LR -> PC, bad return

    .end                @ end of the file
```

We have a few comments regarding the above code:

- We have illustrated two approaches to return to the caller.
  - The first one uses `br lx` as illustrated in `mp_add_in_s`.
  - The second one pops the value preserved for LR into PC as illustrated in `mp_add_in_s_inefficient`.
- There is a bug in `mp_add_in_s_bad_return`, the missing of an effective return. This can be fixed by either changing `pop {r4, lr}` to `pop {r4, pc}` or adding another line of code `bx lr` after the POP instruction.

14.6.2 Header file for ASM functions

To call the above ASM functions in C, we need to prototype them using a header file, as shown in `fc1s_asm_add_in_s.h`:

```
#pragma once

#ifdef __cplusplus
extern "C" {
#endif /* __cplusplus */

int mp_add_in_s(int x, int y);
int mp_add_in_s_inefficient(int x, int y);
int mp_add_in_s_bad_return(int x, int y);

#ifdef __cplusplus
}
#endif /* __cplusplus */
```

14.6.3 Type-casting during function call

There are two approaches to check the type-casing. The first is to use a debugger to halt the code at the first line in the function to check the value of the register. Yet, this is labor intensive.

The second, which is the preferred approach, is to print out the result. To illustrate this, we define a signed 8-bit integer with a value of 0xFE, the TC of -2. Now, we pass this variable to R0 and value 0 to R1 while calling the `int mp_add_in_s(int x, int y)` function. As we return a signed 32-bit integer, we see what is passed to R0 will be returned by R0. Here is the code:

```
    int8_t s_8 = 0xFE;
    int casted = mp_add_in_s(s_8, 0);
    printf("int8_t s_8 = 0x%02X is passed as %d \n", s_8,
       casted);
```

We can use similar approaches to pass variables of different types to the function and see the return values to check the automatic type-casting.

14.6.4 The C code and running results

The C code for experimenting type-casting and ASM function call is shown below, which is saved in `fc1s_mixed_c_n_asm_add_app.c`:

```
void run_app(void) {
    printf("\n\nRunning fc1s main App-----------------\n");

    int8_t s_8 = 0xFE;
    uint8_t u_8 = 0xFE;
    int32_t s_32 = 0xFFFFFFFC;
    uint32_t u_32 = 0xFFFFFFFC;

    int casted = mp_add_in_s(s_8, 0);
    printf("int8_t s_8 = 0x%02X is passed as %d \n", s_8,
       casted);
    casted = mp_add_in_s(u_8, 0);
    printf("uint8_t u_8 = 0x%02X is passed as %d \n", u_8,
       casted);
    casted = mp_add_in_s(s_32, 0);
    printf("int32_t s_32 = 0x%08lX is passed as %d \n", s_32,
       casted);
    casted = mp_add_in_s(u_32, 0);
```

```
    printf("uint8_t u_32 = 0x%08lX is passed as %d \n", u_32, casted);

    int normal_sum = mp_add_in_s(s_8, u_8);
    printf("\nNormal_sum of s_8 and u_8 is %d \n", normal_sum);
    int inefficient_sum = mp_add_in_s_inefficient(s_32, u_8);
    printf("Inefficient_sum of s_32 and u_8 is %d \n", inefficient_sum);
    int bad_sum = mp_add_in_s_bad_return(s_8, u_8);
    printf("Bad_sum of s_8 and u_8 is %d \n", bad_sum);

    while (1);
}
```

The printouts for the type-casting in the above code are shown below:

```
int8_t s_8 = 0xFE is passed as -2
uint8_t u_8 = 0xFE is passed as 254
int32_t s_32 = 0xFFFFFFFD is passed as -3
uint8_t u_32 = 0xFFFFFFFC is passed as -4
```

Here are the conclusions we can get for type-casting from the above printouts:

- If we type-cast from a narrow width date type to a wider one, *all our wishes can be granted*—we can get the exact numbers as intended.
- If we type-cast between signed and unsigned of the same width, there is no data change. We just see the data in a different type.

Note that **if we type-cast from a wide width data type to a narrower one, we lose the most significant bytes**.

With type-casting addressed, we can now interpret the printouts of calling of the three ASM functions in the second part of the code.

```
Normal_sum of s_8 and u_8 is 252
Inefficient_sum of s_32 and u_8 is 251
Bad_sum of s_8 and u_8 is 0
```

Note that a bad return is a common reason causing the program to freeze in ASM programming. Another common reason is the bad logic that causes the program trapped in an endless loop.